

TransferLab

Capabilities and first-experiment guide

Prepared for a new researcher or operator. 1 October 2026.

TransferLab studies how training one LLM ability changes other abilities. For example: after training on cryptographic capture-the-flag tasks, does a model improve on ordinary programming, mathematics, logic, or science questions? Does a benign tool-use curriculum produce similar benefits?

The codebase provides one experiment workflow: freeze the model and datasets, evaluate the unchanged model, train one domain with verifiable rewards, evaluate saved checkpoints, and compare results across domains and seeds.

The default study starts each arm independently from Qwen2.5-7B-Instruct. The smaller SmoLLM2-135M-Instruct model is provided for inexpensive execution checks. The repository is at [LLM-RLVR-Transfer-Learning](#).

Current qualification evidence

Stage	Recorded status	Evidence and meaning
local	PASSED	Regression and fixture workflow checks; 15.2 s elapsed; 64 tests, 0 skipped
cpu	PASSED	Real tiny-model execution and resume; 43.8 s elapsed
sandbox	BLOCKED	Docker isolation and official EvalPlus checks; Docker is missing. Install a working Docker engine and build <code>containers/sandbox.Dockerfile</code>
gpu	NOT EXECUTED	Intended-model, five-arm CUDA pilot

Passing a stage qualifies that stage only. The intended-model GPU pilot is required before funding the paper suite. A running training loop does not by itself demonstrate learning: useful GRPO updates require different rewards among responses to the same prompt.

Who can use this

The local checks need a terminal, Python 3.12, uv, and enough space for PyTorch. Code and tool execution require a working Docker engine. Paper training additionally needs one BF16-capable CUDA GPU, the prepared datasets, and a compatible driver. Vast operations need an account key, SSH authentication, rsync, and an explicitly configured GPU image.

What experiments are implemented?

Five training arms

Arm	Default training data	What it measures
crypto	Random-Crypto tasks from HackSynth-GRPO	Crypto problem solving with Python tools
benign_tools	Generated transformations and debugging sums	Benign computation using the same tools
code	TACO stdin/stdout problems	Writing complete Python programs
math	NuminaMath-TIR problems with boxed solutions	Mathematical problem solving
logic	Uniquely solvable Knights-and-Knaves puzzles	Logical deduction

Each arm has 512 training examples and a separate 64-example validation set in the supplied study configs. Equal dataset counts do not mean equal token counts or equal difficulty. Actual exposure and runtime are recorded separately from the compute estimate.

Common controls

All paper arms share the base-model revision, LoRA settings, GRPO settings, evaluation snapshot, and estimated training-compute target. The default study uses seeds 11, 22, and 33. Evaluations occur before training and at 25%, 50%, and 100% of the compute target. The unchanged model provides each run's paired baseline.

RLVR means that a program checks the answer and supplies a reward. GRPO samples several responses to the same prompt and uses their relative rewards to train the model. LoRA trains small adapter matrices while keeping base weights frozen.

The evaluation suite

Benchmark	Default tasks	Main ability
Verified Random-Crypto	50	Cryptography
Self-contained InterCode subset	5	Encoding and tool use
Benign tools	64	Data transformation and debugging
HumanEval+ and MBPP+	32 each	Programming
MATH500	128	Mathematics
Knights-and-Knaves	64	Logic
ARC-Challenge	64	Science knowledge

The validation set is also evaluated. Optional EleutherAI harness support adds ARC, HellaSwag, or MMLU likelihood scores. Those results have their own output files and interpretation.

Run the initial checks

Run commands from the repository root. Choose a fresh output directory each time; qualification outputs are never overwritten. Every stage saves readiness.json, logs, timestamps, and a source fingerprint. A failed or blocked stage exits nonzero.

1. Local regression and workflow checks

```
uv sync --frozen --extra train --group dev
uv run --frozen --extra train --group dev transferlab \
  initial-tests --stage local --output runs/checks/local
```

This runs the selected regression suite without skipped training tests, then exercises all five arms using marked fixtures. It checks data integrity, evaluation resumption, reporting, tool rewards and loss masking, checkpoint handling, and mocked rental failure/recovery cases. This stage downloads dependencies but needs no model download, Docker engine, or rental.

2. Real tiny-model execution and resume

```
uv run --frozen --extra train transferlab initial-tests \
  --stage cpu --output runs/checks/cpu
```

This downloads and pins SmolLM2-135M-Instruct, evaluates a tiny procedural dataset, performs one real LoRA/GRPO update, saves and reloads the adapter, evaluates it, then resumes the completed run. The command checks that resume leaves predictions unchanged. It forces CPU execution. Zero rewards are possible and qualify execution only.

3. Docker execution and official code verifiers

```
docker pull python:3.11-slim
docker build -f containers/sandbox.Dockerfile \
  -t transferlab-sandbox:0.1.0 .
uv run --frozen --group dev transferlab initial-tests \
  --stage sandbox --output runs/checks/sandbox
```

The sandbox checks known-correct and incorrect code, candidate timeouts, excessive output, a read-only filesystem, disposable tool state, blocked network access, and the official EvalPlus checker. Docker must already be running. Generated candidate code stays inside disposable containers.

The source tree must be present for initial-tests: it locates the configs and tests from --project, which defaults to the current directory. --timeout bounds each subprocess; the default is 7,200 seconds. A timed-out subprocess and its process group are terminated.

Prepare data and qualify the intended GPU

Freeze the five-arm study

Install the paper extras and qualify Docker first. Dataset preparation may download large source datasets even though the selected training subset is small.

```
uv sync --frozen --extra train --extra benchmarks --group dev
uv run --frozen --extra train --extra benchmarks transferlab \
  prepare-suite --output data/study
```

prepare-suite resolves upstream model and dataset revisions, prepares crypto first, and reuses its exact evaluation snapshot for the remaining four arms. Repeating the command verifies and reuses completed preparations with matching configs. Differing or damaged snapshots fail verification.

Preparation rejects overlapping normalized prompts, duplicated task IDs, unsupported callable TACO problems, and redacted Zebra answers. EvalPlus reference answers are computed in Docker and checked before preparation completes. Review Random-Crypto labels and the proposed distance taxonomy before freezing the paper design.

Execute an initial GPU pilot

On the intended Linux GPU runtime:

```
uv run --frozen --extra train --extra benchmarks --group dev \
  transferlab initial-tests --stage gpu \
  --prepared data/study/crypto data/study/benign_tools \
  data/study/code data/study/math data/study/logic \
  --output runs/checks/gpu --steps 2 --tasks 1
```

This runs preflight against each frozen config, then executes a bounded five-arm pilot. Preflight checks the visible CUDA GPU, BF16 support, installed training API, model tokenizer/tool parser, and actual sandbox checker. The runtime must have the selected sandbox images. The Linux lock uses a CUDA 13 PyTorch build; use a compatible driver/runtime.

The small initial pilot verifies execution on the chosen hardware. For the paper feasibility gate, run a larger pilot, inspect rewards and nonzero gradients, and derive a common compute budget:

```
uv run --frozen --extra train --extra benchmarks transferlab \
  pilot data/study/crypto data/study/benign_tools \
  data/study/code data/study/math data/study/logic \
  --output runs/pilot --steps 8 --tasks 4
uv run --frozen transferlab pilot-budget \
  data/study/crypto data/study/benign_tools data/study/code \
  data/study/math data/study/logic \
  --pilots runs/pilot --output data/study/budget.json
```

A rejected pilot means that the recipe, task curation, runtime or budget needs attention. Zero within-group reward variation yields no useful GRPO learning signal. The gate also checks that the projected evaluation cost fits its allocation.

Run the paper suite and inspect results

One command for all arms and seeds

```
uv run --frozen --extra train --extra benchmarks transferlab \
  suite data/study/crypto data/study/benign_tools \
  data/study/code data/study/math data/study/logic \
  --budget data/study/budget.json --output runs/paper
uv run --frozen --extra plots transferlab report runs/paper \
  --output reports/paper --plots
```

The suite checks that all arms share the same model, recipe, study settings, and evaluation files. Each arm and seed gets a separate output directory. Every model starts independently from the frozen base. Add `--resume` to continue interrupted work in the same output layout.

Reading the output files

Location	Contents
data/study/ARM	Frozen config, source revisions, checksummed task splits
runs/paper/ARM/seed-N/run.json	Status, exact config, source/environment provenance
train/rewards.jsonl	Rewards and tool traces
train/compute.jsonl	Estimated compute, tokens, update steps and elapsed time
train/checkpoint-N	Verified full recovery checkpoint or milestone adapter
train/final	Final LoRA adapter and tokenizer
eval/CHECKPOINT	Predictions, manifest and aggregate metrics
reports/paper	Transfer JSON, CSV, Markdown and optional plot

`pass@1` estimates correctness for one sampled response. `pass@4` estimates whether at least one of four sampled responses is correct. A positive transfer delta means the trained model scores higher than its unchanged baseline under the same evaluation settings.

Reports require completed non-pilot runs, every configured seed and checkpoint, and exactly one row per expected task/sample. They verify artifact checksums, prompt/task identity, model/recipe/runtime compatibility and prediction provenance. Partial runs and duplicate samples fail validation.

Task bootstrap intervals summarize uncertainty over benchmark questions. Seed standard deviation is shown separately. Distance-tier averages give each benchmark equal weight. These are descriptive transfer summaries; the software does not automatically establish a causal mechanism or a security-risk conclusion.

Overnight runs on Vast.ai

Set up the controller and runtime

Set `VAST_API_KEY` in the controller environment and register your SSH key with Vast. The runtime needs SSH, `rsync`, `uv/Python`, `GNU timeout`, compatible `CUDA`, and access to a Docker engine capable of running the sandbox. Configure `vast.image` and its immutable `sha256 image_digest` in the experiment YAML after verifying the runtime.

The defaults require one GPU with at least 48 GB VRAM, 98% advertised reliability, 100 GB disk, at most USD 2/hour, USD 30 estimated exposure per job, and a 12-hour rental window. The study allocation is USD 100 pilot, 550 training, 250 evaluation, and 100 contingency.

Inspect an offer before a funded launch

```
uv run --frozen transferlab vast offers configs/crypto.yaml
uv run --frozen transferlab vast run configs/crypto.yaml \
  --data data/study --output runs/remote-pilot \
  --category pilot --dry-run -- \
uv run --frozen --extra train --extra benchmarks transferlab \
  pilot /workspace/data/crypto /workspace/data/benign_tools \
  /workspace/data/code /workspace/data/math /workspace/data/logic \
  --output /workspace/results --steps 8 --tasks 4
```

Remove `--dry-run` only for a funded launch. The runner searches and ranks offers, reserves estimated exposure, persists a creation label, waits for SSH, uploads code/data, executes one bounded command, collects results and verifies instance destruction. New rental attempts are blocked while unresolved recovery state exists.

Collection and recovery

Results are copied to the controller in the background. Transient collection failures retry without terminating training. The final 15 minutes are reserved for retrieval. If retrieval fails, the runner stops the GPU and retains the instance and a recovery record. Storage charges continue while the instance is stopped.

```
uv run --frozen transferlab vast status
uv run --frozen transferlab vast collect configs/crypto.yaml
uv run --frozen transferlab vast cleanup
```

`collect` restarts a stopped instance if needed, retrieves results under a bounded deadline, then destroys it. If collection fails, it stops the GPU again. `cleanup --discard-uncollected` explicitly discards remote results. To resume on a replacement rental, add `--restore runs/remote-pilot` to `vast run` and `--resume` to the remote training command, keeping `/workspace/results` as the output path.

Keep the controller online overnight. The remote timeout stops the process after SSH disconnects, but instance billing still requires provider-side stop/destruction. The budget ledger guards launches using estimates; it is not a provider-enforced account spending cap.

Adaptations and limitations

Changing an experiment

Start from a supplied YAML in configs/. Each config names the model, training and validation sources, shared recipe, evaluation tasks, seeds, sandbox and rental limits. Change the relevant dataset/arm settings, validate, then prepare a fresh snapshot. Freeze those choices before the paper suite.

```
uv run --frozen transferlab validate configs/math.yaml
uv run --frozen --extra train --extra benchmarks transferlab \
  prepare configs/math.yaml --output data/new-math \
  --shared-evaluation data/study/crypto
```

Task adapters accept Hugging Face datasets, pinned GitHub CSV/JSON sources, procedural tasks, local JSONL tasks and EvalPlus datasets. New formats require an adapter in data.py plus an appropriate verifier in tasks.py. Tool arms use execute_python and submit_answer, with bounded calls and disposable Python processes; variables and files do not persist between calls.

Project map

Code	Responsibility
config.py, data.py, tasks.py	Config validation, snapshots, tasks and answer checks
training.py, checkpoints.py	GRPO/LoRA, compute accounting, saving and recovery
episodes.py, tool_parsing.py, sandbox.py	Tool protocol and isolated execution
backends.py, evaluation.py, report.py	Model generation, scoring and transfer reports
initial_tests.py, preflight.py	Initial qualification and runtime checks
vast.py	Rental limits, remote execution, collection and cleanup
cli.py	Command-line entry points

Limits to keep in the paper

Compute is an architecture-based estimate, with declared generation, backward and checkpoint-recomputation costs. It is not profiler-measured FLOPs. Milestones may overshoot by one update; report the residual mismatch.

Default evaluation is on frozen subsets. Crypto tasks and the five inline InterCode tasks do not measure a complete real-world hacking workflow. Random-Crypto training labels are not human verified. The benign curriculum shares the tool interface but has not been established as equally difficult. Public benchmark contamination in base-model pretraining is unknown.

Conceptual-distance labels are researcher annotations. Review and freeze them before training. Full study conclusions require meaningful pilot rewards, all seeds, the declared evaluation suite, and justified statistical interpretation.

Further reading and reproduction

Repository and README: full commands. Upstream recipes and benchmarks: [TRL GRPO](#), [HackSynth-GRPO](#), [InterCode](#), [EvalPlus](#). Retain upstream licenses and attribution when distributing derived data.

PDF built 2026-10-02 05:29 UTC. Repository base commit: 087f26f. Source fingerprint: 5b7700d2294dc5df. Qualification evidence is saved in the corresponding runs/checks output folders.